

Computer Programming (NCSV101) [Winter 2025-26]

Slides 7.3

Unions and Enums in C

SAURABH SRIVASTAVA

ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

IIT (ISM) DHANBAD



What you should know by know ...

An array is a “collection of variables of *same* type”

A structure variable is a “collection of variables of *different* types”

Let us now have a look at some “pseudo collection” of variables as well !!

- The reason we are calling them “pseudo” collections is because they may seem like a collection of variables ...
- ... while behind the scenes, it'll be just one guy with a multiple-personality disorder :D

Union – a collection that isn't a collection

A union in C is created in a similar fashion as a structure

Sample Structure Template

```
struct sample_structure
{
    int i;
    double d;
    char c;
};
```

Sample Union Template

```
union sample_union
{
    int i;
    double d;
    char c;
};
```

There is, however, a drastic difference between the two

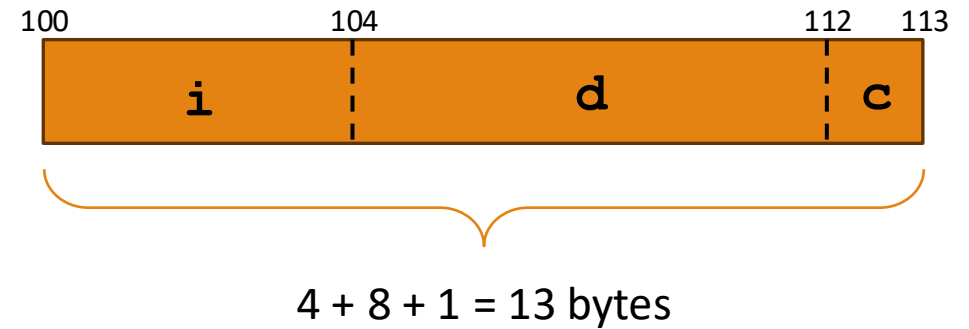
- While a structure is a collection of all its members, a union is just a *façade* over its individual members

At the back, there is not enough storage to store *all* the members

- Instead, there is just enough storage to store *any one* of them
- It means that only the memory large enough to store the *most spacious* member is available in a union

Memory Allocation – Structures vs Unions

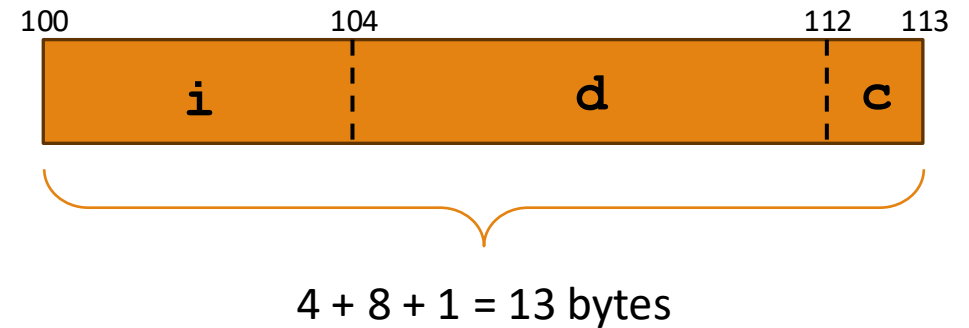
```
struct sample_structure s;
```



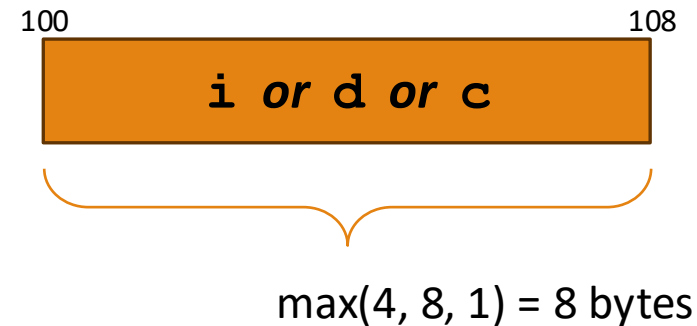
All member variables of a structure variable have their defined space within the allocated memory block

Memory Allocation – Structures vs Unions

```
struct sample_structure s;
```



```
union sample_union u;
```



Just enough storage is allocated so that any one member of the union may be stored

Using Unions

You can use unions in the same fashion as you use structures, using the same operators/conventions

- So, this is absolutely fine ...
`u.d = 6.5;`
- ... and so is this
`printf("%d", u.i);`
- The difference though is that you are writing/reading the same parts of memory in both the statements above
- So, while the first statement “stores” the value 6.5 in bytes 0 to 7 ...
- ... the second statement “interprets” the bytes 0 to 3 as if it is an integer

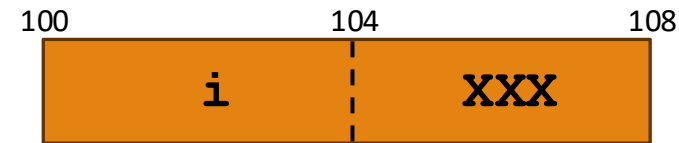
Each read or write uses the same locations

- Thus, a good practice is to “tie” a particular union variable to a particular member, and stick with it
- Otherwise, you may be risking unexpected read or writes

Interpretation of Union Memory

```
union sample_union u;
```

```
printf("%d", u.i);
```



A member's storage starts from the beginning of the block and runs up to its size ...

Interpretation of Union Memory

```
union sample_union u;
```

```
printf("%d", u.c);
```



... with the rest of the storage being ignored for all practical purposes

Interpretation of Union Memory

```
union sample_union u;  
  
printf("%lf", u.d);
```



The optimal use of the space, of course, is when you interpret it in the form of the most spacious member

Using Unions

You can use unions in the same fashion as you use structures, using the same operators/conventions

- So, this is absolutely fine ...
`u.d = 6.5;`
- ... and so is this
`printf("%d", u.i);`
- The difference though is that you are writing/reading the same parts of memory in both the statements above
- So, while the first statement “stores” the value 6.5 in bytes 0 to 7 ...
- ... the second statement “interprets” the bytes 0 to 3 as if it is an integer

Each read or write uses the same locations

- Thus, a good practice is to “tie” a particular union variable to a particular member, and stick with it
- Otherwise, you may be risking unexpected read or writes

Note that the pointer to member or arrow operator is also available for unions

Using in Practice – Structures vs Unions (1/3)

```
#include<stdio.h>

typedef struct sample_struct
{
    int i;
    double d;
    char c;
} str; // Alias "str" created for the template

typedef union sample_union
{
    int i;
    double d;
    char c;
} un; // Alias "un" created for the template
```

We are creating a structure and a union with identical templates here !

Using in Practice – Structures vs Unions (2/3)

```
int main()
{
    str s;
    printf("Size of s: %zu\n", sizeof(s));
    s.i = 5;
    printf("[A]. Printing s: %d, %lf, %c\n", s.i, s.d, s.c);
    s.d = 6.5;
    printf("[B]. Printing s: %d, %lf, %c\n", s.i, s.d, s.c);
    s.c = 'd';
    printf("[C]. Printing s: %d, %lf, %c\n", s.i, s.d, s.c);

    un u;
    printf("Size of u: %zu\n", sizeof(u));
    u.i = 5;
    printf("[A]. Printing u: %d, %lf, %c\n", u.i, u.d, u.c);
    u.d = 6.5;
    printf("[B]. Printing u: %d, %lf, %c\n", u.i, u.d, u.c);
    u.c = 'd';
    printf("[C]. Printing u: %d, %lf, %c\n", u.i, u.d, u.c);

    return 0;
}
```

Using in Practice – Structures vs Unions (2/3)

```
int main()
{
    str s;
    printf("Size of s: %zu\n", sizeof(s));
    s.i = 5;
    printf("[A]. Printing s: %d, %lf, %c\n", s.i, s.d, s.c);
    s.d = 6.5;
    printf("[B]. Printing s: %d, %lf, %c\n", s.i, s.d, s.c);
    s.c = 'd';
    printf("[C]. Printing s: %d, %lf, %c\n", s.i, s.d, s.c);

    un u;
    printf("Size of u: %zu\n", sizeof(u));
    u.i = 5;
    printf("[A]. Printing u: %d, %lf, %c\n", u.i, u.d, u.c);
    u.d = 6.5;
    printf("[B]. Printing u: %d, %lf, %c\n", u.i, u.d, u.c);
    u.c = 'd';
    printf("[C]. Printing u: %d, %lf, %c\n", u.i, u.d, u.c);

    return 0;
}
```

First, we assign values one-by-one to all members of the structure and see if the values are retained or not ...

Using in Practice – Structures vs Unions (2/3)

```
int main()
{
    str s;
    printf("Size of s: %zu\n", sizeof(s));
    s.i = 5;
    printf("[A]. Printing s: %d, %lf, %c\n", s.i, s.d, s.c);
    s.d = 6.5;
    printf("[B]. Printing s: %d, %lf, %c\n", s.i, s.d, s.c);
    s.c = 'd';
    printf("[C]. Printing s: %d, %lf, %c\n", s.i, s.d, s.c);

    un u;
    printf("Size of u: %zu\n", sizeof(u));
    u.i = 5;
    printf("[A]. Printing u: %d, %lf, %c\n", u.i, u.d, u.c);
    u.d = 6.5;
    printf("[B]. Printing u: %d, %lf, %c\n", u.i, u.d, u.c);
    u.c = 'd';
    printf("[C]. Printing u: %d, %lf, %c\n", u.i, u.d, u.c);

    return 0;
}
```

... and then, we do the same with the union as well !!

Using in Practice – Structures vs Unions (3/3)

```
[saurabhsrivastava@Mac For Lecture 7.3 % gcc union-vs-structure.c -o uvs  
[saurabhsrivastava@Mac For Lecture 7.3 % ./uvs  
Size of s: 24  
[A]. Printing s: 5, 0.000000,  
[B]. Printing s: 5, 6.500000,  
[C]. Printing s: 5, 6.500000, d  
Size of u: 8  
[A]. Printing u: 5, 0.000000,  
[B]. Printing u: 0, 6.500000,  
[C]. Printing u: 100, 6.500000, d  
saurabhsrivastava@Mac For Lecture 7.3 %
```

Using in Practice – Structures vs Unions (3/3)

```
saurabhsrivastava@Mac For Lecture 7.3 % gcc union-vs-structure.c -o uvs
saurabhsrivastava@Mac For Lecture 7.3 % ./uvs
Size of s: 24
[A]. Printing s: 5, 0.000000,
[B]. Printing s: 5, 6.500000,
[C]. Printing s: 5, 6.500000, d
Size of u: 8
[A]. Printing u: 5, 0.000000,
[B]. Printing u: 0, 6.500000,
[C]. Printing u: 100, 6.500000, d
saurabhsrivastava@Mac For Lecture 7.3 %
```

Ignoring the size part, which seems too much as compared to our theoretical expectations (we will discuss it shortly), convince yourself that the changes to one member does not have any side effects on other members

Using in Practice – Structures vs Unions (3/3)

```
saurabhsrivastava@Mac For Lecture 7.3 % gcc union-vs-structure.c -o uvs
saurabhsrivastava@Mac For Lecture 7.3 % ./uvs
Size of s: 24
[A]. Printing s: 5, 0.000000,
[B]. Printing s: 5, 6.500000,
[C]. Printing s: 5, 6.500000, d
Size of u: 8
[A]. Printing u: 5, 0.000000,
[B]. Printing u: 0, 6.500000,
[C]. Printing u: 100, 6.500000, d
saurabhsrivastava@Mac For Lecture 7.3 %
```

For the union, the size is as expected – only 8 bytes which is just enough to accommodate a double value

Using in Practice – Structures vs Unions (3/3)

```
saurabhsrivastava@Mac For Lecture 7.3 % gcc union-vs-structure.c -o uvs
saurabhsrivastava@Mac For Lecture 7.3 % ./uvs
Size of s: 24
[A]. Printing s: 5, 0.000000,
[B]. Printing s: 5, 6.500000,
[C]. Printing s: 5, 6.500000, d
Size of u: 8
[A]. Printing u: 5, 0.000000,
[B]. Printing u: 0, 6.500000,
[C]. Printing u: 100, 6.500000, d
saurabhsrivastava@Mac For Lecture 7.3 %
```

For the union, the size is as expected – only 8 bytes which is just enough to accommodate a double value

The same memory block is “interpreted” differently, when you read or assign some value to a particular member

But still, you are guaranteed that the “interpretation” will be as per expectations, if you set and read, the same member !!

Digression – Padding for Performance

When we create a structure variable, the compiler may insert *extra unused bytes* between its members

- These bytes are called *padding*
- Padding ensures that each member is stored at a memory address that meets its *alignment requirements*
- Typically it is used for better read/write performance allowing the CPU to access data more efficiently

The alignment requirement for a particular member is dependent on the member's size

- Usually, a member of size k bytes, must be stored at an offset divisible by k bytes within the structure's block
- Thus, if the structure's block starts at address 100 , the member can be stored starting at 100 , $100+k$, $100+2k$ etc.
- For a member of type `double`, the possible addresses will 100 , 108 , 116 and so on
- If the previous member ended at an address which is not going well with this alignment, ...
- ... some extra bytes are added as padding

Note that there are padding requirements for the whole structure block too !!

- A structure block must be stored at a starting address divisible by n , where n is the size of the *largest* member
- In our case, it means that the size of any variable of type `str` shall be a multiple of 8 (owing to `double d`)

Actual Memory Allocation with Padding

```
struct sample_structure s;
```

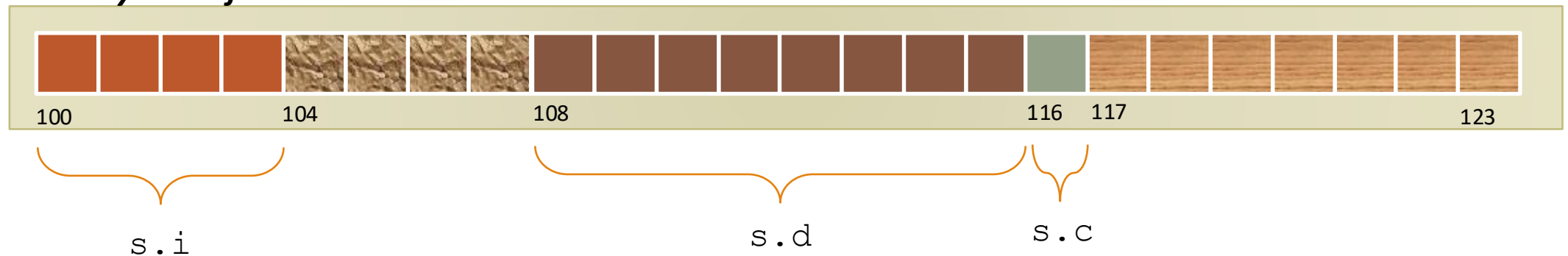
Memory Block for s



Actual Memory Allocation with Padding

```
struct sample_structure s;
```

Memory Block for s



Padding to fulfil alignment requirements of `s.d`



Padding to fulfil alignment requirements of `s`

The other “pseudo-collections” – Enums

As you may now understand, a union is not really a collection – it is essentially just one value

- The storage taken up (besides the padding) by a union variable is essentially the storage of the largest member
- The alignment requirements for such a variable is the “strictest of alignment requirements out of all the members”
- Check out the Resources for the Lecture for a more detailed discussion on the same

Imagine if all the members of the union are of the same type, in particular, `int` !

- An `enum` may be seen as a “union with only integer members”

Another way to look at an enum is a “collection of integer constants” for the same use case

- For example, `SUNDAY` or `MONDAY` are two of the seven possible constant values for “Day of the Week”
- We can create an `enum` to represent these constants:

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN };  
enum Day today;  
today = WED;
```
- Internally, `MON` is a constant for value 0, `TUE` is the same for 1 and so on with `SUN` implying 6

Using in Practice – Enums (1/3)

```
#include<stdio.h>

typedef enum Day_of_Week { MON, TUE, WED, THU, FRI, SAT, SUN } DAY;
// We can now use DAY as an alias for this enum

DAY nextDay(DAY today)
{
    return (today + 1) % 7;
}

DAY prevDay(DAY today)
{
    return (7 + today - 1) % 7;
}
```

Here, we are creating an enum, which is essentially a casing to store an integer, that can store any one out of the seven possible values defined for it

Using in Practice – Enums (1/3)

```
#include<stdio.h>

typedef enum Day_of_Week { MON, TUE, WED, THU, FRI, SAT, SUN } DAY;
// We can now use DAY as an alias for this enum

DAY nextDay(DAY today)
{
    return (today + 1) % 7;
}

DAY prevDay(DAY today)
{
    return (7 + today - 1) % 7;
}
```

Here, we are creating an enum, which is essentially a casing to store an integer, that can store any one out of the seven possible values defined for it

We can now, seamlessly use it across functions as well making the code more readable

Be careful when you perform any operations over an enum, since the values may easily go “out of range” (hence, the % op here, **for the subtraction, we are adding 7 first, so that the range remains valid and does not become negative**)

Using in Practice – Enums (2/3)

```
int main()
{
    DAY today = SUN, yesterday, tomorrow;

    printf("Today is Sunday, it is represented as %d\n", SUN);
    printf("Tomorrow is Monday, it represented as %d\n", MON);
    printf("Yesterday was Saturday, it represented as %d\n", SAT);

    tomorrow = nextDay(today);
    yesterday = prevDay(today);
    printf("Enum representations for the all three: ");
    printf(" %d, %d, %d\n", today, tomorrow, yesterday);

    printf("The size of the enum: %zu\n", sizeof(today));
    return 0;
}
```

You can now use DAY like a new type, although, in the background it is just an `int`

Using in Practice – Enums (3/3)

```
saaurabhsrivastava@Mac For Lecture 7.3 % gcc days-as-enum.c -o dae
saaurabhsrivastava@Mac For Lecture 7.3 % ./dae
Today is Sunday, it is represented as 6
Tomorrow is Monday, it represented as 0
Yesterday was Saturday, it represented as 5
Enum representations for the all three: 6, 0, 5
The size of the enum: 4
saaurabhsrivastava@Mac For Lecture 7.3 %
```

Notice that the Constants like SUN and MON are assigned actual constant integer values

The size of the enum is just the size of an int

Homework !!

Create a program using a union to store student marks as either `int` or `float`

Additional Reading

Go through the Resources for the lecture, in particular, a file called *Padding Concepts.pdf*